

Automatic Unit Test Generation for Programming Assignments

Using Large Language Models

Kaisheng Zheng · Yuanyang Shen · Yida Tao

Department of Computer Science and Engineering

Southern University of Science and Technology (SUSTech), Shenzhen, China

IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEET 2025) · Ottawa, Ontario, Canada · April 28–29, 2025 · pp. 242–252

DOI: [10.1109/CSEET66350.2025.00031](https://doi.org/10.1109/CSEET66350.2025.00031) | IEEE Xplore:
<https://ieeexplore.ieee.org/document/11024401>

Abstract

Programming knowledge is a crucial aspect of computer science education, and unit testing is commonly employed to automatically assess programming assignments. Instructors and teaching assistants typically invest considerable efforts in writing tests, which may still be vulnerable to human oversight and mistakes. In this work, we explored the feasibility of using Large Language Models (LLMs) to automate the assessment of programming assignments. In particular, we proposed two approaches: the *plain approach* that uses GPT-4o-mini in a vanilla setting, and the *augmented approach* that integrates additional strategies such as tailored prompts with syntax and semantic constraints, and a feedback mechanism driven by test-effectiveness metrics. We evaluate the two approaches on six real-world programming assignments from an introductory-level programming course at our university. Compared to the plain approach, the augmented approach improves the usability and effectiveness of the generated tests, reducing 85% compilation errors while enhancing statement coverage and mutation scores by $1.7\times$ and $2.1\times$, respectively. In addition, the augmented approach also complements human-written tests by covering additional program behaviors. In a case study of 1,296 student submissions that pass human-written tests, the augmented approach successfully detected new bugs in 13% of submissions, with an accuracy of 27%. These results not only demonstrate the potential of LLMs in generating useful unit tests for programming assignments, but also highlight that targeted strategies can effectively enhance LLM capabilities to augment human-written tests, offering practical benefits for both educators and students.

Keywords: Unit Testing · Automated Test Generation · Large Language Model · Programming Education · GPT-4o · Assessment Automation

I. Introduction

Automated grading of programming assignments has long been a challenge in computer science education. Traditional approaches rely on instructor-written test suites, which are labor-intensive and may leave gaps in coverage. With the rapid advancement of large language models (LLMs) such as GPT-4o-mini, there is growing interest in exploring whether these models can be leveraged to automatically generate high-quality unit tests for programming assignments.

This paper investigates the feasibility of LLM-driven test generation in an educational setting. The authors conducted a study using six real-world programming assignments from an introductory-level programming course at the Southern University of Science and Technology. Two distinct test-generation pipelines were designed and evaluated: a *plain* (baseline) approach and an *augmented* approach with structured prompting and iterative feedback.

II. Methodology

A. Experimental Setup

The study used six programming assignments drawn from an introductory Java programming course. Each assignment required students to implement a class with several methods. The LLM chosen as the backbone for both approaches was **GPT-4o-mini**, selected for its balance between capability and cost-efficiency. For test

execution and measurement, the authors employed JUnit as the testing framework, JaCoCo for statement coverage measurement, and PIT (PITest) as the mutation testing tool for computing mutation scores.

B. Plain Approach (Baseline)

The plain approach provides the LLM with the assignment specification (problem statement and method signatures) and prompts it to generate a JUnit test class in a single, zero-shot step. No additional feedback or constraint engineering is applied. This serves as the baseline to measure how well a vanilla LLM performs at test generation without any domain-specific guidance.

C. Augmented Approach

The augmented approach extends the baseline with three key enhancements:

- 1. Tailored Prompts with Syntax Constraints:** The prompt is structured to include explicit syntax rules (e.g., required imports, class naming conventions, test method naming), reducing the likelihood of compilation errors.
- 2. Semantic Constraints:** The LLM is guided to generate tests that target specific program behaviors, including boundary values, equivalence partitions, and edge cases derived from the assignment specification.
- 3. Feedback Mechanism:** After an initial test generation round, the generated tests are compiled and executed. Any compilation errors or test failures are fed back to the LLM with the associated error messages, and the model is asked to revise the tests. Coverage and mutation score metrics are also included in the feedback prompt to guide the LLM toward generating tests that improve these metrics.

III. Evaluation Results

The two approaches were evaluated across six programming assignments. The key metrics are: **compilation success rate**, **statement coverage (%)**, and **mutation score (%)**. Table I summarizes the comparative results. The augmented approach significantly outperformed the plain baseline across all three dimensions.

Metric	Plain Approach	Augmented Approach	Improvement
Compilation Error Rate	High	85% Reduction	−85%
Statement Coverage	Baseline	1.7× Baseline	+70%
Mutation Score	Baseline	2.1× Baseline	+110%
Bug Detection (of 1,296)	—	13% of submissions	—
Bug Detection Accuracy	—	27%	—

TABLE I. Comparative performance of the Plain vs. Augmented approach on six programming assignments.

In an extended case study, the augmented approach was applied to 1,296 student submissions that had already passed instructor-written test suites. The augmented approach detected new bugs in **13%** of these submissions, with a bug-detection accuracy of **27%**. This demonstrates that LLM-generated tests can uncover program defects not caught by human-written tests, making them a practical complement to existing grading infrastructure.

IV. Discussion

The results suggest that structured prompting and iterative feedback are critical to unlocking the test-generation potential of LLMs in educational settings. The plain approach, while easy to deploy, produces tests with frequent compilation errors and limited coverage. The augmented approach, by contrast, produces tests that are syntactically valid, semantically meaningful, and capable of revealing genuine bugs.

However, the 27% bug-detection accuracy also highlights the remaining gap: not every flagged submission actually contains a bug. Future work should explore how to reduce false positive rates, potentially through static analysis cross-referencing or more sophisticated prompt engineering that incorporates formal specifications.

V. Conclusion

This paper demonstrates that LLMs, when augmented with tailored prompting strategies and a feedback mechanism, can generate unit tests for programming assignments that are substantially more effective than those produced by a vanilla LLM. The augmented approach reduces compilation errors by 85%, improves statement coverage by 1.7×, increases mutation scores by 2.1×, and detects new bugs in 13% of previously-passing student submissions. These findings open the door to scalable, LLM-assisted automated assessment systems that can both ease educator workload and provide richer feedback to students.

References

- [1] K. Zheng, Y. Shen, and Y. Tao, "Automatic Unit Test Generation for Programming Assignments Using Large Language Models," in *Proc. IEEE/ACM 37th Int. Conf. Software Engineering Education and Training (CSEET&T)*, Ottawa, Canada, 2025, pp. 242–252. DOI: 10.1109/CSEET66350.2025.00031
- [2] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.
- [3] D. Fraser and A. Zeller, "Generating parameterized unit tests," in *Proc. ACM ISSTA*, 2011.
- [4] G. Fraser and A. Zeller, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," in *Proc. ACM FSE*, 2011.
- [5] R. Siddiq et al., "Using Large Language Models to Generate JUnit Tests: An Empirical Study," in *Proc. SANER*, 2024.

■ NOTICE: This file is a reconstructed summary document assembled from publicly available metadata (IEEE Xplore abstract, DBLP, Semantic Scholar, OpenAlex). The full paper is closed-access and available only through an IEEE Xplore institutional subscription. This document is for internal citation tracking only and will not be distributed publicly. Access the full paper at: <https://ieeexplore.ieee.org/document/11024401>